面向长时程在线三维重建的分层记忆系统研究计划

基于 TTT3R、VGGT、OVGGT、MERG3R 与 LoGoPlanner 思想的统一方案

CVPR Final Project Proposal

2026-04-05

摘要

本项目聚焦于**长时程在线三维重建**，目标是在固定显存与计算预算下，使系统在长序列、重复结构和重访场景中保持更稳定的几何一致性，而不是将项目包装成完整的端到端 SLAM 或导航系统。我们将当前仓库中的 TTT3R 视为在线重建主干，其优势在于通过测试时更新规则改善长序列长度泛化；在此基础上，本文提出一个新的分层记忆方案 **HMR3D**：使用 TTT3R 风格的在线 active-state 作为实时主路径，使用 VGGT 风格的局部高质量几何求解器提升短程精度，使用 OVGGT 风格的重要性缓存与 anchor 保护控制预算，使用 MERG3R 风格的子图归档与恢复机制建立可重访的长期记忆，并将 LoGoPlanner 的 query-based geometry readout 思想用于 token 汇总、描述子构造和子图摘要生成。

本文首先明确在线三维重建与 SLAM 的边界：我们不以全局导航和闭环控制为目标，而是以**持续建图、重访恢复、全局消歧和预算受限条件下的长程稳定性**为核心问题。随后，本文给出完整的方法定义，包括状态表示、token 类型、短程层、长程活动层、归档层、路由器机制、事件触发策略、最小可行实现路径和扩展版本；同时提出一套完整的评测协议，既保留 TTT3R 官方的 pose、depth 和 reconstruction 基线评测，也引入 Horizon Scaling、Branch Disambiguation、Forced Backtracking 与 Revisit/Loop-closure 等针对长期记忆能力的 stress tests。

我们预计该方案的主要贡献在于：将在线重建从单一 recurrent state 扩展为可写入、可保留、可归档、可恢复的分层记忆系统；建立适合在线重建而非纯导航的长程评测协议；并给出一条能够在当前 TTT3R 代码库中逐步落地、风险可控、可做消融验证的实现路线。

目录

1 项目定位与问题定义 **4**

1.1 项目边界：在线三维重建不是完整 SLAM 4
1.2 项目目标 4
1.3 为什么当前单一在线状态仍然不够 5

2 相关工作与设计原则 5
2.1 截至 2026-04-05 的关键相关工作 5
2.2 五条设计原则 6

3 整体方案：HMR3D 7
3.1 方法概述 7
3.2 状态定义 7
3.3 token 类型设计 8
3.4 短程层：VGGT 风格局部高精度 + LoGoPlanner 风格查询读出 9
3.4.1 短程层结构 9
3.4.2 输入编码与局部工作流 9
3.4.3 VGGT 在短程层中的作用 10
3.5 活动层：TTT3R 风格写入 + OVGGT 风格预算控制 10
3.5.1 活动层结构 10
3.5.2 TTT3R 风格的写入规则 11
3.5.3 OVGGT 风格的重要性评分与 anchor 保护 11
3.6 归档层：MERG3R 风格子图组织、检索与恢复 12
3.6.1 子图表示 12
3.6.2 LoGoPlanner 风格的摘要形成 12
3.6.3 归档触发与恢复触发 12
3.6.4 MERG3R 在归档层中的作用 13
3.7 路由器：统一管理写入、保留、归档与恢复 13
3.8 五条技术线在本项目中的最终分工 14

4 两级实现路线：最小可行版本与完整版本 14
4.1 最小可行版本（建议先完成） 14
4.2 完整版本（作为扩展目标） 15
4.3 表示层策略：先解耦系统问题，再考虑高斯实例化 15

5 训练与优化策略 16
5.1 阶段一：训练无关改造 16
5.2 阶段二：若需要再引入训练目标 16

6 评测方案 **17**
6.1 TTT3R 已经证明了什么 17
6.2 基础 benchmark 与标准指标 17
6.3 四组长期记忆 stress tests 17
6.3.1 任务组 1：Horizon Scaling 18
6.3.2 任务组 2：Branch Disambiguation 18
6.3.3 任务组 3：Forced Backtracking（由 Hidden Detour 翻译而来） . 18
6.3.4 任务组 4：Revisit / Loop-closure 19
6.4 消融实验设计 19
6.5 为什么不把 ROS bag 或 IsaacLab 作为主评测 20
6.6 成功标准 20

7 在当前代码库中的落地方案 **20**
7.1 现有工程基础 20
7.2 建议新增的模块文件 21
7.3 分阶段实现计划 21
7.4 最先应完成的三件事 21

8 风险分析与应对策略 **22**
8.1 主要风险 22
8.2 风险缓解 22

9 预期贡献与最终交付 **22**
9.1 学术贡献 22
9.2 工程交付 23

10 结论 **23**

1 项目定位与问题定义

1.1 项目边界：在线三维重建不是完整 SLAM

本项目的核心任务是**在线三维重建 / 在线建图**，而不是完整的 SLAM 系统。为了避免概念混淆，表 1 对两者的目标与评测重点进行区分。

表 1: 在线三维重建与 SLAM 的区别

维度	在线三维重建	SLAM
核心目标	持续恢复场景几何、深度、点云或稠密结构	同时估计位姿并维护可用于定位的地图
输出重点	depth、point map、point cloud、submap、dense latent map	trajectory、relocalization、loop closure、global map
主要约束	长序列下的几何一致性、预算受限的持续推理	实时定位稳定性、重定位、回环、闭环控制接口
常见失败模式	长序列漂移、历史信息遗忘、重复结构混淆、局部几何退化	丢跟踪、重定位失败、回环错误、拓扑错乱
典型指标	深度误差、点云精度、完整度、法向一致性、长程漂移曲线	ATE、RPE、追踪成功率、回环收益、重定位成功率
本项目定位	主轴在在线重建，借鉴 SLAM 中子图、重访、长期记忆思想	不是完整目标，只借鉴其长期地图组织机制

因此，最准确的项目表述不是“我们做了一个新 SLAM”，而是：

> 我们研究的是 **SLAM 启发的长时程在线三维重建**，目标是在固定预算下提升长期几何一致性、全局消歧能力和重访恢复能力。

1.2 项目目标

本项目的最终成果不应定义为“接入了更多模块”或“切换了某种地图表示”，而应定义为系统层面的可验证能力提升：

1. 在**固定显存与计算预算**下，随着 horizon 增长，性能退化速度更慢。
2. 在**重复走廊、相似门口、对称分叉**等局部高相似场景中，更不容易发生全局混淆。
3. 在**重访与回到旧区域**时，能够通过历史子图检索与恢复降低累计漂移。
4. 在**在线流式模式**下工作，而不是依赖离线全局批处理。

1.3 为什么当前单一在线状态仍然不够

TTT3R 已经表明，仅通过修改测试时状态更新规则，就能显著提升 CUT3R 这类 recurrent 3D reconstruction foundation model 的长度泛化能力。然而，这类方法仍然主要依赖**单一活动状态**来同时承担以下任务：

- 表示最近观测的局部细节；
- 保留长期历史信息；
- 在预算受限时进行压缩；
- 在重访时恢复历史结构。

这四类职责具有内在冲突。局部细节要求状态保持高分辨率；长期记忆要求状态具有稳定性；预算控制要求状态被压缩；重访恢复又要求历史信息可检索且可注入。把所有职责压缩进同一个 recurrent state，通常会带来三类问题：

1. **长期信息覆盖**：新的观测不断写入时，早期区分性线索会被覆盖。
2. **局部相似导致的全局混淆**：若缺乏 persistent global memory，重复结构下容易误把新区域当旧区域，或把旧区域当新区域。
3. **预算受限时没有明确的保留原则**：若不显式区分 anchor、context 与 archived submap，就很难回答“哪些必须保留，哪些可以删除，哪些应该归档”。

因此，本项目的核心假设是：

> 在线三维重建的长程问题，本质上不是只靠“更强 backbone”解决，而是要把单一在线状态升级为**分层记忆系统**。

2 相关工作与设计原则

2.1 截至 2026-04-05 的关键相关工作

表 2 总结了当前最相关的几条技术路线，以及本项目将从中具体借用什么。

表 2: 关键相关工作与本项目的借鉴方式

工作	已核对时间	论文核心点	本项目借鉴方式
TTT3R	2025-09-30 提交，2026-03-03 修订	将在线重建视为测试时在线学习问题，基于 alignment confidence 给出 closed-form update rate	作为实时主干的 active-state 写入规则
VGGT	2025-03-14 提交	用 feed-forward 模型直接输出相机、point map、depth 和 point track	作为短程高质量局部重建器或其设计原型
Streaming 4D VGGT	2025-07-15 提交，2026-03-31 修订	因果 transformer + historical KV cache 的流式重建	说明局部流式高质量模块是可行的，但 cache 仍需受控
OVGGT	2026-03-06 提交，2026-03-09 修订	Self-Selective Caching + Dynamic Anchor Protection，实现常数级成本流式几何推理	提供训练无关的 keep/evict 与 anchor 保护原则
MERG3R	2026-03-02 提交	reorder、partition、overlap、global alignment 与 confidence-weighted BA 的 divide-and-conquer 框架	作为归档子图的 merge/recovery 机制
LoGoPlanner	2025-12-22 提交，2025-12-23 修订	metric-aware geometry memory + query-based readout，用隐式几何记忆服务规划	不借其 planner head，只借其 query-based token aggregation 与 geometry readout

2.2 五条设计原则

综合上述工作，本项目采用以下设计原则：

1. **在线主路径必须简单稳定。**实时主干不应依赖复杂的全局批处理,因此保留 TTT3R/CUT3R 风格的递归更新主路径。

2. **短程高精度与长程可持续应分开处理。**短程问题适合 VGGT 类强局部几何推理，长程问题适合 persistent state 和 archive/retrieve 机制。

3. **预算控制优先使用训练无关的显式规则。**在项目早期，不引入黑盒 learned router，而是优先采用 OVGGT 风格的重要性剪枝和 anchor 保护。

4. **长期记忆应以子图而非单一 latent 组织。**归档层采用 MERG3R 风格的 submap 思想，避免把所有历史信息压成不可恢复的单状态。

5. **LoGoPlanner 只用于“如何读和汇总几何记忆”。**不使用其导航头，不把其 latent policy 作为本项目主目标。

3 整体方案：HMR3D

3.1 方法概述

我们将提出一个新的分层记忆系统 **HMR3D**（Hierarchical Memory Reconstruction for 3D）。系统的主流程如下：

1. 输入当前帧 x_t 及其基础几何先验，编码为 observation tokens。

2. 进入**短程层**，结合最近窗口与局部 anchors，得到局部几何摘要、位姿摘要和重访描述子。

3. 由**路由器**判断当前信息应如何处理：写入活动状态、晋升为 anchor、归档为子图，或直接丢弃。

4. **活动层**在 TTT3R 风格写入规则下持续更新，同时使用 OVGGT 风格的重要性评分和 anchor 保护控制预算。

5. 当出现场景切换、预算压力或重访触发时，**归档层**把活动子图压缩为可检索子图，并在需要时按 MERG3R 风格进行对齐、合并和恢复。

6. **LoGoPlanner 风格 query tokens** 贯穿于短程读出、子图摘要与重访检索三个阶段，用于把几何记忆转化为可用的摘要，而不是原样传递所有 token。

3.2 状态定义

在时间步 t，系统状态定义为

$$\mathcal{S}_t = \{M^t_{\text{short}},\ M^t_{\text{active}},\ B^t_{\text{bank}}\}, \tag{1}$$

其中：

- M^t_{short} 为短程局部工作区，负责最近观测的高保真建模；
- M^t_{active} 为当前活动区域的 persistent online state；

- B^t_{bank} 为已归档的历史子图库，可检索、可恢复。

给定当前观测 x_t，系统还将计算若干中间量：

$$\Delta_t,\ c^t_{\text{align}},\ s^t_{\text{import}},\ d_t,\ u_t,\ q^t_{\text{pose}},\ q^t_{\text{geo}},\ q^t_{\text{desc}},\ q^t_{\text{sum}}, \tag{2}$$

其中：

- Δ_t 表示候选状态增量；
- c^t_{align} 表示新观测与当前记忆的一致性；
- s^t_{import} 表示 token 重要性；
- d_t 表示当前窗口的重访检索描述子；
- u_t 表示当前局部的歧义/不确定性强度；
- 四类 query token 分别用于位姿读出、几何读出、检索读出与摘要读出。

3.3 token 类型设计

本项目不再把所有信息都视为“同一种 token”，而是明确区分五类 token，如表 3 所示。

表 3: 五类 token 的职责划分

符号	名称	作用
P_t	observation tokens	来自当前帧的 patch/ray/point-level 观测 token，保留丰富局部信息，但生命周期较短
A_t	anchor tokens	从 observation 中晋升得到的局部稳定锚点，承载对位置、边界、结构或重访判别有帮助的线索
H_t	active tokens	活动层 persistent state 的基础单元，对应当前在线地图中的长期状态
Z_j	archived summary tokens	子图归档后的压缩摘要 token，用于索引、检索与恢复
Q	query tokens	用于按任务读取记忆的查询 token，包括 q_{pose}、q_{geo}、q_{desc}、q_{sum}

3.4 短程层：VGGT 风格局部高精度 + LoGoPlanner 风格查询读出

3.4.1 短程层结构

短程层的目标不是替代活动层，而是用有限窗口保存最近观测的高分辨率几何细节。其结构定义为

$$M^t_{\text{short}} = \{L^t_{\text{recent}},\ A^t_{\text{short}},\ Q_{\text{short}}\}, \tag{3}$$

其中：

- L^t_{recent} 为最近 W 帧的 live tokens；
- A^t_{short} 为窗口内部已晋升的局部 anchors；
- Q_{short} 为用于局部读出的 query tokens 集合。

3.4.2 输入编码与局部工作流

每个时间步中，短程层执行以下过程：

1. **编码：**

$$P_t = \text{Enc}(x_t, r_t, \pi_t), \tag{4}$$

其中 r_t 为几何或射线先验，π_t 为位姿/尺度相关先验。

2. **追加到短程缓存：**将 P_t 写入 L^t_{recent}。

3. **LoGoPlanner 风格 query-based readout：**

$$z^t_{\text{pose}} = \text{Attn}(q_{\text{pose}}, L^t_{\text{recent}} \cup A^t_{\text{short}}), \tag{5}$$

$$z^t_{\text{geo}} = \text{Attn}(q_{\text{geo}}, L^t_{\text{recent}} \cup A^t_{\text{short}}), \tag{6}$$

$$d_t = \text{Proj}\big(\text{Attn}(q_{\text{desc}}, L^t_{\text{recent}} \cup A^t_{\text{short}})\big)\,. \tag{7}$$

这一步的核心思想来自 LoGoPlanner：**不是把所有历史 token 原样传给下游，而是让不同 query 从几何记忆中按任务读取摘要。**

4. **anchor 晋升：**当某些 observation token 在多帧支持下表现出较高区分性与稳定性时，将其晋升为 anchor：

$$m_{\text{promote}}(i) = \mathbb{I}[s_{\text{import}}(i) > \tau_s \wedge \text{support}(i) > \tau_k]. \tag{8}$$

5. **局部驱逐：**对短期 live tokens 执行受预算约束的删减，确保窗口成本稳定。

3.4.3 VGGT 在短程层中的作用

VGGT 在本项目中不作为全局 backbone，而作为**事件触发的局部高质量几何求解器**。我们只在以下情况下调用 VGGT 或 VGGT 风格局部 refine：

- 短程窗口累计到固定关键帧数；
- 当前局部歧义强，u_t 上升；
- 当前置信度下降，怀疑活动状态无法正确整合新信息；
- 出现疑似重访，需要用高质量局部几何进行验证。

VGGT 局部模块的输出包括：

- 局部高精度相机与 point map；
- 更稳定的局部几何摘要 z^t_{geo}；
- 更可区分的重访描述子 d_t；
- 对候选增量 Δ_t 的局部修正。

这意味着 VGGT 的职责是“**把最近这段看清楚**”，而不是“接管长程在线记忆”。

3.5 活动层：TTT3R 风格写入 + OVGGT 风格预算控制

3.5.1 活动层结构

活动层是系统的实时主路径，定义为

$$M^t_{\text{active}} = \{A^t_{\text{active}},\ C^t_{\text{active}}\}, \tag{9}$$

其中：

- A^t_{active} 表示较慢更新、偏稳定的 anchor state；
- C^t_{active} 表示较快更新、偏适应性的 context state。

这种双速率设计的动机是：若所有长期状态以同样速度更新，要么过快导致遗忘，要么过慢导致无法适应新观测。

3.5.2 TTT3R 风格的写入规则

我们保留 TTT3R 的核心思想：用新观测与当前记忆的一致性来控制写入强度，而不是使用固定更新率。设基础写入掩码为 m^t_{base}，则实际写入率为

$$m^t_{\text{write}} = m^t_{\text{base}} \cdot \text{clip}(c^t_{\text{align}}, 0, 1). \tag{10}$$

在此基础上，对 anchor 与 context 使用不同更新速率：

$$A^{t+1}_{\text{active}} = (1 - \alpha_{\text{anchor}} m^t_{\text{write}}) \odot A^t_{\text{active}} + \alpha_{\text{anchor}} m^t_{\text{write}} \odot \Delta^t_{\text{anchor}}, \tag{11}$$

$$C^{t+1}_{\text{active}} = (1 - \alpha_{\text{ctx}} m^t_{\text{write}}) \odot C^t_{\text{active}} + \alpha_{\text{ctx}} m^t_{\text{write}} \odot \Delta^t_{\text{ctx}}, \tag{12}$$

并约束

$$\alpha_{\text{anchor}} < \alpha_{\text{ctx}}. \tag{13}$$

直观上，anchor state 更保守，context state 更灵活。这样既能继承 TTT3R 的长度泛化优势，也能减少单状态被不断覆盖的问题。

3.5.3 OVGGT 风格的重要性评分与 anchor 保护

活动层不是无限扩张的。我们在此引入 OVGGT 风格的训练无关缓存维护原则，为每个 token 计算重要性分数：

$$\text{score}_i = \lambda_r r_i + \lambda_a a_i + \lambda_g g_i + \lambda_c c_i + \lambda_v v_i, \tag{14}$$

其中：

- r_i 表示 recency；
- a_i 表示是否为 anchor 或是否被多次 query 读出；
- g_i 表示几何支撑强度，例如跨帧支持、重投影稳定性；
- c_i 表示置信度；
- v_i 表示该 token 在重访验证、分支消歧等关键事件中的参与程度。

基于该分数，路由器对 token 执行四类动作：

$$\text{action}_i \in \{\text{keep}, \text{promote}, \text{archive}, \text{evict}\}. \tag{15}$$

同时，对被判定为 coordinate-critical 的 anchor 执行保护，即便总体预算紧张也不优先删除。这一设计借鉴 OVGGT 的 Dynamic Anchor Protection，其意义在于：

> 在线几何系统中最不应该被预算机制误删的，不是“最老的 token”，而是“最能维持坐标与身份稳定的 token”。

3.6 归档层：MERG3R 风格子图组织、检索与恢复

3.6.1 子图表示

归档层不是简单地把活动状态扔掉，而是把已经离开的区域组织成可检索子图。每个归档子图定义为

$$B_j = \{d_j,\ z_j,\ A_j,\ T_j,\ w_j,\ \mu_j\}, \tag{16}$$

其中：

- d_j 是检索描述子；
- z_j 是压缩后的子图摘要；
- A_j 是子图中的关键 anchors；
- T_j 是子图局部坐标变换与锚定位姿；
- w_j 是子图置信度与可靠性统计；
- μ_j 是元信息，如时间段、场景编号、重访次数、覆盖范围等。

3.6.2 LoGoPlanner 风格的摘要形成

归档不是做简单平均池化，而是使用 query-based summarization。对即将关闭的活动子图，先形成

$$U_j = \{S^j_{\text{active}},\ A^j_{\text{active}},\ T_j\}, \tag{17}$$

再用摘要 query 计算：

$$z_j = \text{Pool}_Q(q_{\text{sum}}, U_j), \qquad d_j = \text{Proj}(z_j). \tag{18}$$

这里，LoGoPlanner 的思想再次发挥作用：**摘要应由任务相关查询读出，而非无差别压缩。**

3.6.3 归档触发与恢复触发

归档触发可定义为

$$\mathcal{T}_{\text{archive}}(t) = \mathbb{I}[b_t > \tau_b \vee o_t < \tau_o \vee a_t > \tau_a], \tag{19}$$

其中：

- b_t 表示预算压力；
- o_t 表示当前区域与活动层核心区域的 overlap；

- a_t 表示活动子图的年龄或生命周期。

重访恢复触发可定义为

$$\mathcal{T}_{\text{retrieve}}(t,j) = \mathbb{I}\left[\cos(d_t, d_j) > \tau_r \wedge \text{GeoVerify}(t,j) = 1\right]. \tag{20}$$

一旦触发恢复，执行如下流程：

1. 使用当前描述子 d_t 从 B_{bank} 中检索 top-k 候选子图；
2. 结合局部几何与位姿一致性进行验证；
3. 将通过验证的 (A_j, z_j) 注入当前活动层作为 prior；
4. 在当前短程窗口上做一次局部 refine；
5. 必要时触发 MERG3R 风格的 overlap alignment 与 confidence-weighted merge。

3.6.4 MERG3R 在归档层中的作用

MERG3R 在本项目里不被视作每帧运行的 backbone，而是**后台地图整理器**。它解决的是：

- 当单一活动窗口已经离开某区域时，如何把该区域压成一个可管理的子图；
- 当多个历史子图存在 overlap 时，如何做全局对齐与合并；
- 当重访发生时，如何以子图为单位恢复，而不是从零开始重新估计。

因此，MERG3R 的职责是“**把很长的历史组织起来并在需要时找回来**”。

3.7 路由器：统一管理写入、保留、归档与恢复

路由器并非一开始就设计为完全学习得到的大黑盒，而是一个**规则优先、可逐步学习化**的控制模块。其输入包括：

- 对齐置信度 c^t_{align}；
- token 重要性分数 s^t_{import}；
- 当前局部歧义度 u_t；
- 预算压力 b_t；
- 重访候选得分；
- 场景切换与 overlap 统计。

路由器输出四类决策：

1. **write**：是否将当前候选增量写入活动层；
2. **keep/evict**：活动 token 如何裁剪；
3. **archive**：是否关闭当前活动子图并归档；
4. **retrieve**：是否从归档层恢复历史子图。

本项目早期采用显式事件触发规则：

- 若 c^t_{align} 高且预算允许，则优先写入；
- 若 u_t 高，则触发短程局部 refine；
- 若 b_t 高，则优先执行 keep/evict/anchor protection；
- 若检索相似度高且几何验证通过，则触发 retrieve；
- 若已明显离开当前区域，则触发 archive。

3.8 五条技术线在本项目中的最终分工

表 4: TTT3R、VGGT、OVGGT、MERG3R 与 LoGoPlanner 的职责划分

技术线	在本项目中的职责
TTT3R	负责在线主路径的 active-state 更新规则，解决“怎么稳定写入”
VGGT	负责短程窗口内的高质量局部几何推理，解决“最近这段怎么更准确”
OVGGT	负责预算受限下的 keep/evict 与 anchor protection，解决“哪些必须留下，哪些可以丢”
MERG3R	负责归档子图的组织、对齐、合并与重访恢复，解决“历史怎么存得下、找得回”
LoGoPlanner	负责 query-based token readout 与 summary formation，解决“记忆怎么被读成有用的局部状态、描述子和摘要”

4 两级实现路线：最小可行版本与完整版本

4.1 最小可行版本（建议先完成）

考虑到项目周期与工程复杂度，本项目的第一阶段不建议一开始就集成全部外部模型，而应先完成一个**最小可行版本**：

1. 保留当前 TTT3R backbone 与 checkpoint；
2. 增加结构化的状态表示与路由器；
3. 引入 OVGGT 风格的重要性评分和 anchor 保护；
4. 引入最简单的 submap bank；
5. 实现基于 q_{desc} 与 q_{sum} 的归档、检索和恢复；
6. 在现有官方评测基础上加入长程 stress tests。

该版本的优点是：

- 不需要重新训练大模型；
- 能较早得到可验证的消融结果；
- 能直接回答“persistent memory 是否真的有必要”。

4.2 完整版本（作为扩展目标）

在最小可行版本稳定后，再逐步扩展到完整版本：

1. 引入实际的 VGGT 局部模块或其等价局部多视图 refine；
2. 在归档层加入更接近 MERG3R 的 overlap alignment 与 confidence-weighted merge；
3. 若需要，再尝试把规则路由器逐步替换为轻量 learned router；
4. 进一步探索表示层解耦，例如将分层记忆实例化到 Gaussian submap。

4.3 表示层策略：先解耦系统问题，再考虑高斯实例化

本项目的主问题是**记忆系统设计**，不是场景表示本身。因此方法应该与表示层解耦：

- 第一阶段以 point map / depth / latent state 为主；
- 第二阶段可以选择性地实例化到 Gaussian submap；
- 不建议在项目早期同时切换到新的 GS 优化框架，否则会把重点从“长期记忆”转移到“渲染与 densification 细节”。

换言之，本项目的命题应当是：

> 我们不是在提出新的表示，而是在提出一种**适用于在线三维重建的分层记忆系统**；Gaussian 只是未来可能的载体之一。

5 训练与优化策略

5.1 阶段一：训练无关改造

项目初期尽量坚持训练无关策略，原因有三：

- 可以最大限度复用 TTT3R 已有 checkpoint；
- 便于将改进归因于状态管理而不是重新训练；
- 更容易做干净的消融对比。

此阶段主要依赖：

- TTT3R 风格写入率；
- OVGGT 风格重要性评分与 anchor 保护；
- MERG3R 风格子图组织与对齐；
- LoGoPlanner 风格 query-based summary。

5.2 阶段二：若需要再引入训练目标

如果项目后期需要进一步提升性能，可定义联合损失：

$$\mathcal{L}_{\text{total}} = \lambda_{\text{pose}}\mathcal{L}_{\text{pose}} + \lambda_{\text{depth}}\mathcal{L}_{\text{depth}} + \lambda_{\text{recon}}\mathcal{L}_{\text{recon}} + \lambda_{\text{rev}}\mathcal{L}_{\text{rev}} + \lambda_{\text{budget}}\mathcal{L}_{\text{budget}}, \tag{21}$$

其中：

- $\mathcal{L}_{\text{pose}}$ 约束局部和全局位姿稳定；
- $\mathcal{L}_{\text{depth}}$ 约束逐帧深度质量；
- $\mathcal{L}_{\text{recon}}$ 约束点云/重建精度与完整度；
- $\mathcal{L}_{\text{rev}}$ 约束重访恢复后的一致性；
- $\mathcal{L}_{\text{budget}}$ 约束活动层和归档层的容量。

在本 proposal 中，我们不把重新训练设为第一目标；重新训练只作为扩展选项。

6 评测方案

6.1 TTT3R 已经证明了什么

TTT3R 官方评测已经提供了三条成熟基线：

1. **Camera Pose Estimation**：以 ATE、RPE trans、RPE rot 为核心指标；
2. **Video Depth**：以 Abs Rel、Sq Rel、RMSE、Log RMSE、$\delta < 1.25$ 等为核心指标；
3. **3D Reconstruction**：以 Acc、Comp、NC 等点云与表面一致性指标为核心指标。

这说明本项目的主 quantitative evaluation 不需要先搬到 ROS 或 IsaacLab 才成立。相反，最稳妥的路线是：

> 先在公开数据集上证明“同一 backbone 下，仅靠更好的记忆维护就能改善长程退化”，再考虑仿真或 bag 级 demo。

6.2 基础 benchmark 与标准指标

表 5: 建议保留的基础 benchmark

任务	数据集	指标
Pose / RelPose	ScanNet、TUM-dynamics、Sintel	ATE、RPE trans、RPE rot、每 100 帧漂移率
Video Depth	KITTI、Bonn、Sintel	Abs Rel、Sq Rel、RMSE、Log RMSE、$\delta < 1.25$
3D Reconstruction	7Scenes、可扩展到 NRGBD	Acc、Comp、NC、FPS、显存占用

除了平均值，本项目必须重点报告**指标随序列长度变化的曲线**，因为我们的主张不是短序列精度，而是长序列退化更慢。

6.3 四组长期记忆 stress tests

为了回答“persistent memory 是否有必要”，我们将把老师给出的四类导航式测试翻译为适用于在线重建的实验协议。

6.3.1 任务组 1：Horizon Scaling

设置：在相同 simulator 或相同真实场景分布下，将序列切成不同 horizon 档位，例如 50、100、200、400、800、1000 帧，或按实际路径长度分成 4–10m、10–20m、20–40m、40m+。

检验问题：系统性能是否会随着序列增长出现系统性退化。

主要指标：

- ATE / RPE 随序列长度的曲线；
- depth RMSE 随序列长度的曲线；
- reconstruction Acc / Comp 随序列长度的曲线；
- peak VRAM、active token 数、bank 大小、FPS。

希望证明的结论： **HMR3D** 的退化斜率小于 CUT3R/TTT3R baseline。

6.3.2 任务组 2：Branch Disambiguation

设置：构造重复走廊、相似房门、镜像岔路等局部视野高度相似而全局身份不同的场景。

检验问题：固定容量历史压缩是否会丢失早期关键线索，导致全局身份混淆。

主要指标：

- branch confusion rate；
- wrong relocalization rate；
- top-k retrieval recall；
- 分支后 ATE / RPE；
- 分支后地图错配率。

希望证明的结论：引入 submap archive + retrieve 后，系统在重复结构下的全局消歧能力更强。

6.3.3 任务组 3：Forced Backtracking（由 Hidden Detour 翻译而来）

设置：轨迹先远离目标区域，再绕行返回；或者人为截取“先背离、后回归”的观测序列。

检验问题：系统是否只是依赖最近局部信息，还是能够在绕行后维持长期几何一致性。

注意：由于本项目不是 planner，这一任务不应评估“规划是否最优”，而应评估**绕行轨迹下的建图与定位一致性**。

主要指标：

- 远离后返回时的 ATE / RPE；
- 绕行过程中的 depth drift；
- 返回后重访恢复是否使误差回落；
- 子图检索是否成功命中旧区域。

6.3.4 任务组 4：Revisit / Loop-closure

设置：要求轨迹多次穿越相似区域，或经过某区域后再回到该区域。

检验问题： Persistent memory 是否对长距离重访具有不可替代性。

主要指标：

- revisit retrieval recall；
- 恢复触发率与误触发率；
- 恢复前后 ATE / RPE / depth drift 改善幅度；
- loop consistency gain；
- identity consistency。

6.4 消融实验设计

为了保证结论可信，至少需要以下消融：

1. **CUT3R baseline**；
2. **TTT3R baseline**；
3. **TTT3R + anchor protection**；
4. **TTT3R + submap bank**；
5. **TTT3R + submap bank + retrieval**；
6. **HMR3D 完整版**；
7. 去掉 query-based summary，改用简单平均池化；
8. 去掉 archive，仅保留 active-state；

9. 去掉 anchor protection，仅做 budget 剪枝；
10. 去掉 VGGT 局部 refine，仅依赖短程缓存。

6.5 为什么不把 ROS bag 或 IsaacLab 作为主评测

本项目的主张是“在线重建的长期记忆与子图恢复”，不是“闭环控制质量”。因此：

- **公开数据集应该是主 quantitative 评测。**这样便于和 TTT3R 的官方口径一致，也便于追踪精确 GT。
- **ROS bag 更适合做 demo，不适合做主证据。**很多 bag 缺乏稳定的 dense GT，适合展示，不适合做主表格。
- **IsaacLab 可用于后期构造可控 stress test。**如果需要批量生成重复走廊、强重访、强绕行序列，IsaacLab 是可选项，但不应在项目早期成为阻塞项。

6.6 成功标准

本项目的成功不以“是否引入了更多模块”评判，而以以下结果为标准：

1. 在 long-horizon 序列上，**HMR3D** 的 ATE / RPE / depth drift 退化曲线明显优于 TTT3R baseline；
2. 在 Branch Disambiguation 与 Revisit 任务上，retrieve/recover 机制带来可测改善；
3. 在固定预算下，VRAM 与 active token 数保持有界增长；
4. 消融能够证明每个模块的作用，而不是所有提升都来自更重的计算。

7 在当前代码库中的落地方案

7.1 现有工程基础

当前仓库中已经具备以下优势：

- TTT3R 主干与 checkpoint 已经就绪；
- 官方 pose / depth / reconstruction 评测脚本已经存在；
- 现有代码已经有 model update type 的切换逻辑，适合做状态更新方案对比。

因此，项目不需要从零搭建系统，而应围绕现有代码做结构化扩展。

7.2 建议新增的模块文件

建议在 `CVPR/TTT3R/src/dust3r/` 下增加以下模块：

- `hier_state.py`：定义 M_{short}、M_{active}、B_{bank} 的数据结构；
- `memory_router.py`：实现 write / keep / archive / retrieve 的规则路由器；
- `submap_bank.py`：实现归档子图的存储、检索、恢复；
- `revisit.py`：实现 q_{desc}、几何验证和恢复注入；
- `importance.py`：实现 OVGGT 风格的重要性评分与 anchor protection；
- `local_refine.py`：实现短程局部 refine 接口，先用轻量版本，后续再接入 VGGT 风格模块。

7.3 分阶段实现计划

表 6: 建议的六阶段工程计划

阶段	目标	具体工作
P1	跑通基线评测	复现实验脚本，记录 TTT3R 在不同 horizon 下的曲线，建立对照组
P2	状态结构化	引入 `hier_state.py` 与基础路由器，显式区分 short / active / bank
P3	预算控制	实现 importance scoring、anchor protection 与 keep/evict 逻辑
P4	子图归档与恢复	实现 `submap_bank.py`、粗检索、几何验证、恢复注入
P5	长程 stress tests	实现 Horizon Scaling、Branch、Forced Backtracking、Revisit 评测脚本
P6	局部 refine 扩展	接入 VGGT 风格局部 refine，完善消融与最终报告

7.4 最先应完成的三件事

从项目管理角度看，最应优先完成的是：

1. **先有 eval，再做改动**。若没有长程评测曲线，继续改代码没有判断依据。
2. **先做 submap bank 原型，再考虑更强局部模块**。因为 persistent memory 的必要性首先要在 Revisit/Branch 上被验证。

3. **先用规则路由，再考虑 learned router**。项目早期不建议把不确定性又引入控制层。

8 风险分析与应对策略

8.1 主要风险

1. **系统过大，无法在课程周期内完成**。如果同时接入 TTT3R、VGGT、MERG3R、LoGoPlanner、GS 表示，项目会迅速失控。
2. **改了代码但没有明确收益**。若没有 long-horizon 与 revisit 评测，系统可能越改越复杂但无法证明改进。
3. **路由器过于拍脑袋**。如果控制逻辑没有依据，结果很可能不稳定。
4. **实际问题不在重访，而在局部几何本身**。若 baseline 失败主要来自局部几何太差，那么先做记忆系统也可能收效有限。

8.2 风险缓解

- 用**最小可行版本**控制项目规模；
- 先用 **TTT3R 官方评测 + 四组 stress tests** 建立证据链；
- 路由器优先采用 **TTT3R + OVGGT** 风格的规则组合，而不是大黑盒；
- 若发现局部几何是瓶颈，再增量接入 VGGT 局部 refine，而不是一开始就做大集成。

9 预期贡献与最终交付

9.1 学术贡献

若项目按计划完成，预期贡献可以表述为：

1. 提出一个**面向长时程在线三维重建的分层记忆框架**，将单一 recurrent state 扩展为短程层、活动层与归档层。
2. 将 **TTT3R 的 confidence-aware 写入、OVGGT 的预算控制、MERG3R 的子图组织、LoGoPlanner 的 query-based readout** 统一到一个在线建图系统中。
3. 建立一套**适用于在线重建的长期记忆评测协议**，重点覆盖 Horizon Scaling、Branch Disambiguation、Forced Backtracking 与 Revisit/Loop-closure。

9.2 工程交付

最终交付物应至少包括：

- 一个可以在当前 TTT3R 仓库中运行的分层记忆原型；
- 一套可重复运行的 long-horizon 与 revisit 评测脚本；
- 消融实验结果与最终曲线；
- 一份说明项目边界、方法细节、实验设计和实现计划的完整报告。

10 结论

本 proposal 的核心立场是：**当前在线三维重建真正缺的是长期记忆组织能力，而不是再多接一个语义头。** TTT3R 已经证明测试时状态更新规则可以显著提升长度泛化，但单一活动状态仍无法同时承担局部精度、长期保留、预算受限压缩和重访恢复四类职责。因此，我们提出 **HMR3D** 这一分层记忆方案：用 TTT3R 保持实时主路径，用 VGGT 处理局部高精度，用 OVGGT 进行预算控制与 anchor 保护，用 MERG3R 管理子图归档与恢复，并用 LoGoPlanner 的 query-based 思路把几何记忆读成可用的局部状态、检索描述子和摘要表示。

如果该方案在官方 benchmark 与四组 stress tests 上得到验证，那么它的意义并不在于“做了一个更复杂的系统”，而在于证明了一件更基础的事情：

> 在线三维重建要想真正走向长时程，不仅要会重建，还要会**记住、保留、压缩、找回**。

参考文献与在线资源

1. TTT3R: 3D Reconstruction as Test-Time Training. arXiv:2509.26645. https://arxiv.org/abs/2509.26645
2. VGGT: Visual Geometry Grounded Transformer. arXiv:2503.11651. https://arxiv.org/abs/2503.11651
3. Streaming 4D Visual Geometry Transformer. arXiv:2507.11539. https://arxiv.org/abs/2507.11539
4. OVGGT: O(1) Constant-Cost Streaming Visual Geometry Transformer. arXiv:2603.05959. https://arxiv.org/abs/2603.05959

5. MERG3R: A Divide-and-Conquer Approach to Large-Scale Neural Visual Geometry. arXiv:2603.02351. https://arxiv.org/abs/2603.02351

6. LoGoPlanner: Localization Grounded Navigation Policy with Metric-aware Visual Geometry. arXiv:2512.19629. https://arxiv.org/abs/2512.19629